SOFTWARE REQUIREMENTS SPECIFICATION

ONLINE QUIZ SYSTEM

Version 1.0 | 12 October 2026

Document information	Details
Project	Online Quiz System
Course / Department	Software Engineering
Prepared by	Batchu Hari Nainika (PES1UG24AM461) — SRS; Avyukth KV (PES1UG24AM460) — SAD; Samvith Vaddiparthi (PES1UG24AM468) — STP
Faculty guide	Arpitha K
Status	Submission draft

Purpose: Define verifiable software, security, usability and CI/CD requirements for a browser-based quiz platform.

Document control

Revision history

Version	Date	Description
1.0	12 Oct 2026	Initial consolidated SRS

Approvals

Role	Name	Signature / Date
Team representative	Batchu Hari Nainika	Pending
Faculty guide	Arpitha K	Pending

Table of contents

1. Introduction 3
2. Overall Description 3
3. External Interface Requirements 4
4. System Features (Detailed) 4
5. Non-Functional Requirements 7
6. Quality Attributes and Acceptance Tests 8
7. System Models and Diagrams 9
8. Requirements Traceability Matrix (RTM) 10
Appendix A. Suggested project structure 10
Appendix B. Team collaboration 10

Document convention: FR = functional requirement; NFR = non-functional requirement; SEC = security requirement; AT = acceptance test. “Shall” denotes a mandatory requirement.

1. Introduction

1.1 Purpose

This SRS defines the expected behavior and measurable acceptance conditions of the Online Quiz System. It is intended to guide implementation, testing, reviews and CI/CD evaluation.

1.2 Scope

The system allows students to register, log in, discover quizzes, attempt a quiz once, view scores and quiz-specific leaderboards, and submit feedback. Faculty and administrators create and manage quizzes. Quiz submissions are scored automatically. Delivery includes an automated GitHub Actions CI pipeline.

Out of scope: paid examinations, live video proctoring, question-generation AI, offline quiz attempts and native mobile apps.

1.3 Intended audience

Student developers, faculty evaluators, testers, administrators and maintainers.

1.4 Definitions and acronyms

Term	Meaning
CI/CD	Continuous Integration / Continuous Delivery
JWT	JSON Web Token for authenticated sessions
API	Application Programming Interface
RBAC	Role-Based Access Control
RTM	Requirements Traceability Matrix
Attempt	One student submission for a specific quiz

2. Overall Description

2.1 Product perspective

A responsive React single-page application communicates with a Node.js/Express REST API, which stores user, quiz, question, attempt and feedback data in MongoDB.

2.2 Major functions

Authentication; role-based dashboards; quiz authoring and publication; topic browsing; timed attempts; autosubmission; score computation; personal score history; per-quiz leaderboard; feedback and contact pages; automated CI validation.

2.3 User classes

Role	Capabilities
Student	Browse and attempt published quizzes; view own scores and leaderboards
Faculty	Create, edit, publish and manage owned quizzes
Administrator	Manage quizzes and user roles; moderate system content

2.4 Operating environment

Modern desktop/mobile browsers; Node.js LTS server; MongoDB Atlas or compatible MongoDB instance; GitHub repository and GitHub Actions.

2.5 Constraints and assumptions

Internet access is required. Quiz timers are enforced by the server, not solely by browser state. Each student has one attempt record per quiz; an active attempt can be resumed before the server deadline, but a finalized attempt cannot be repeated. Team members contribute across frontend, backend and database tasks.

3. External Interface Requirements

3.1 User interfaces

Pages: Home, Register, Login, Quiz Topics, Take Quiz, My Scores, Quiz Leaderboard, About, Contact and Feedback. Faculty/admin users additionally access Quiz Management. All pages use consistent navigation, readable form labels, status messages and responsive layouts.

3.2 Hardware interfaces

No specialized hardware is required. Standard browser-capable laptop, desktop or phone.

3.3 Software interfaces

Interface	Purpose
React / Vite	Frontend rendering and build
Express REST API	Authentication, quizzes, attempts and leaderboard endpoints
MongoDB / Mongoose	Persistence and data validation
GitHub Actions	Automated install, lint, test, build and audit
JWT / bcrypt	Session authorization and password hashing

3.4 Communication interfaces

Frontend exchanges JSON with HTTPS REST endpoints. Requests requiring authentication include a bearer token. Secrets are stored in environment variables, never committed to source control.

4. System Features (Detailed)

4.1 Accounts and access

FR-001 The system shall allow new students to register with name, email and password.

Acceptance: Valid data creates a student account; duplicate email is rejected.

FR-002 The system shall authenticate registered users and establish an authorized session.

Acceptance: Correct credentials succeed; incorrect credentials fail without revealing the password.

FR-003 The system shall provide logout and invalidate the user session/token for subsequent protected API requests.

Acceptance: After logout, protected requests using the logged-out session are rejected.

Acceptance: Protected API requests without a valid token return unauthorized.

FR-004 The system shall enforce student, faculty and administrator permissions on the server.

Acceptance: Students cannot invoke faculty/admin quiz-management operations.

4.2 Discovery and navigation

FR-005 The system shall display published quizzes grouped by topic.

Acceptance: Student sees only published quizzes and their topics.

FR-006 The system shall show quiz title, topic, duration and question count before an attempt.

Acceptance: Quiz detail page displays metadata accurately.

FR-007 The system shall provide Home, About, Contact and Feedback pages.

Acceptance: All pages are reachable from navigation and render on mobile widths.

4.3 Quiz authoring and management

FR-008 Faculty/admin shall create quizzes with title, topic, duration and questions.

Acceptance: Valid quiz is saved as a draft.

FR-009 Faculty/admin shall add multiple-choice questions with answer options and a stored correct option.

Acceptance: Questions save and validate at least two options.

FR-010 Faculty/admin shall edit and delete draft quizzes they are authorized to manage.

Acceptance: Changes persist; unauthorized edits are rejected.

FR-011 Faculty/admin shall publish or unpublish a quiz.

Acceptance: Only published quizzes appear in student discovery.

FR-012 The system shall validate required quiz fields before publication.

Acceptance: Incomplete quizzes cannot be published.

4.4 Quiz attempt lifecycle

FR-013 A student shall have at most one attempt record per quiz; an active attempt may be resumed until its deadline, but a finalized attempt cannot be restarted.

Acceptance: Concurrent start requests create at most one active attempt; a finalized attempt blocks another start.

Acceptance: A second attempt is blocked by the API.

FR-014 The system shall present one or more questions with selectable answers and a visible countdown.

Acceptance: Selected answers remain visible while navigating within an attempt.

FR-015 The system shall preserve selected answers during the active attempt.

Acceptance: Saved responses survive a page refresh where supported by the active session.

FR-016 The system shall submit an attempt when the student selects Submit.

Acceptance: Submission is recorded once and a score is returned.

FR-017 The system shall automatically finalize an attempt when its server-enforced time limit expires.

Acceptance: Only answers saved by the deadline are scored.

FR-018 The system shall reject answer updates after final submission or expiry.

Acceptance: Late updates do not change a finalized score.

4.5 Scoring, history and leaderboard

FR-019 The system shall calculate scores automatically using stored answer keys.

Acceptance: Score equals the count of correct responses according to the quiz scoring rule.

FR-020 Students shall see their score after submission, without correct answers or explanations.

Acceptance: Result page and student APIs omit answer keys and explanations.

FR-021 Students shall view a history of their own completed attempts.

Acceptance: My Scores lists quiz title, score and submission date.

FR-022 The system shall maintain a separate leaderboard for each quiz.

Acceptance: Leaderboard contains only attempts for the selected quiz.

FR-023 The leaderboard shall rank attempts by score with a deterministic tie-break rule.

Acceptance: Equal scores are ordered by earlier submission timestamp.

4.6 Feedback and administration

FR-024 Authenticated users shall submit feedback with a rating or message.

Acceptance: Valid feedback is stored and acknowledged.

FR-025 Administrators shall be able to view submitted feedback.

Acceptance: Feedback view is inaccessible to students.

FR-026 Administrators shall manage user roles through protected operations.

Acceptance: Only admins can change role assignments.

4.7 Build and CI workflow

FR-027 Each feature shall be developed on a feature branch and reviewed through a pull request.

Acceptance: Repository history shows branches, PRs and descriptive commits.

FR-028 CI shall run automatically on pushes and pull requests.

Acceptance: GitHub Actions workflow starts for configured events.

FR-029 CI shall execute linting, formatting checks, automated tests and production build.

Acceptance: Pipeline fails when a mandatory check fails.

FR-030 CI shall publish test/coverage reports as downloadable artifacts.

Acceptance: A completed run exposes test and coverage artifacts.

FR-031 CI shall enforce the configured coverage gate and run dependency security checks.

Acceptance: Below-threshold coverage or a configured blocking audit finding fails CI.

5. Non-Functional Requirements

5.1 Performance and reliability

NFR-001 The system shall respond to 95% of normal API requests within 2 seconds under a test load of 50 concurrent users.

Acceptance: Load test records p95 response time at or below 2 seconds.

NFR-002 Quiz finalization shall be idempotent to avoid duplicate submissions.

Acceptance: Repeated submission requests produce one stored final attempt.

NFR-003 Quiz expiry shall be validated against server time.

Acceptance: Manipulating the browser clock cannot extend the allowed duration.

5.2 Usability and maintainability

NFR-004 Core pages shall work at viewport widths of 375 px and above without horizontal scrolling.

Acceptance: Responsive UI inspection passes.

NFR-005 Forms shall provide labels and actionable validation messages.

Acceptance: Required-field errors are announced and visible.

NFR-006 The project shall include setup instructions, environment variable examples and API documentation.

Acceptance: A new developer can install and run the project from README instructions.

NFR-007 Automated test coverage shall meet at least 80% statement coverage for the agreed application-code scope, reported separately for frontend and backend.

Acceptance: Both coverage reports meet the configured threshold, or any exception is explicitly approved and documented.

Acceptance: Coverage report meets the CI threshold.

5.3 Security

5.3.1 Security objectives

SO-01: Protect user accounts, quiz answer keys and administrative operations against unauthorized access.

SO-02: Preserve integrity and fairness of attempts, deadlines, scores and leaderboards.

5.3.2 Security requirements

SEC-001 Passwords shall be stored as salted cryptographic hashes, not plaintext.

Acceptance: Database inspection finds no plaintext passwords.

SEC-002 Protected endpoints shall validate JWT signatures and authorization roles.

Acceptance: Invalid/expired tokens and insufficient roles are rejected.

SEC-003 Student-facing responses shall never expose correct options or scoring keys.

Acceptance: API response tests verify answer-key fields are absent.

SEC-004 The API shall validate and sanitize user-controlled input.

Acceptance: Malformed inputs fail validation without server crashes.

SEC-005 Application secrets and database credentials shall be loaded from environment variables.

Acceptance: Repository scan finds no committed credentials.

SEC-006 Dependency security auditing shall be part of CI.

Acceptance: CI produces audit results and applies documented failure policy.

6. Quality Attributes and Acceptance Tests

Acceptance is based on repeatable automated checks and a short manual demonstration. All high-priority functional and security requirements must pass; the main branch must have a passing CI run.

Test ID	Scenario	Expected outcome
AT-01	Register, login, logout	Valid user can access dashboard; invalid credentials rejected
AT-02	Student attempts quiz twice	First finalized attempt saved; second start denied
AT-03	Quiz timer expires	Attempt finalized with saved answers only
AT-04	Student checks result	Score shown; correct answers hidden
AT-05	Open two quiz leaderboards	Each shows only its own quiz attempts
AT-06	Student calls admin API	403 Forbidden response
AT-07	Faculty publishes incomplete quiz	Validation rejects publication
AT-08	Push failing test to feature branch	CI run fails
AT-09	Push passing implementation	Lint, test, coverage, audit and build pass
AT-10	Inspect responsive UI	No horizontal overflow at 375 px

6.1 Proposed testing tools

Vitest for JavaScript unit tests, React Testing Library for UI behavior, Supertest for Express endpoints, and a controlled MongoDB test database for integration tests. ESLint, Prettier and npm audit run in GitHub Actions.

6.2 CI/CD acceptance criteria

The repository contains a documented workflow triggered by pushes and pull requests. CI installs dependencies, checks code style, executes tests, evaluates coverage, builds the frontend, performs dependency auditing and uploads reports. Merge into main occurs only after review and required checks pass.

7. System Models and Diagrams

7.1 Student use-case diagram

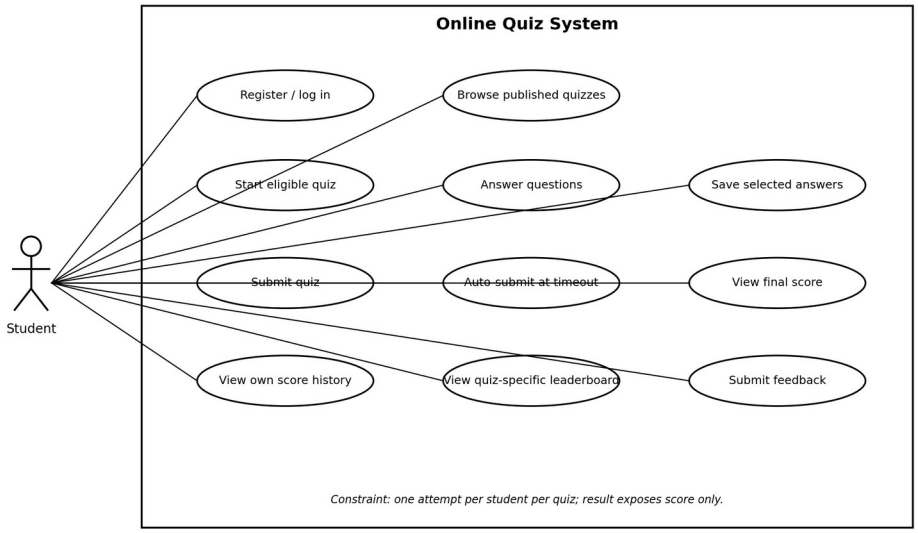


Figure 1. Student interactions: authentication, browsing, taking quizzes, viewing scores and leaderboards, and submitting feedback.

7.2 Faculty and administrator use-case diagram

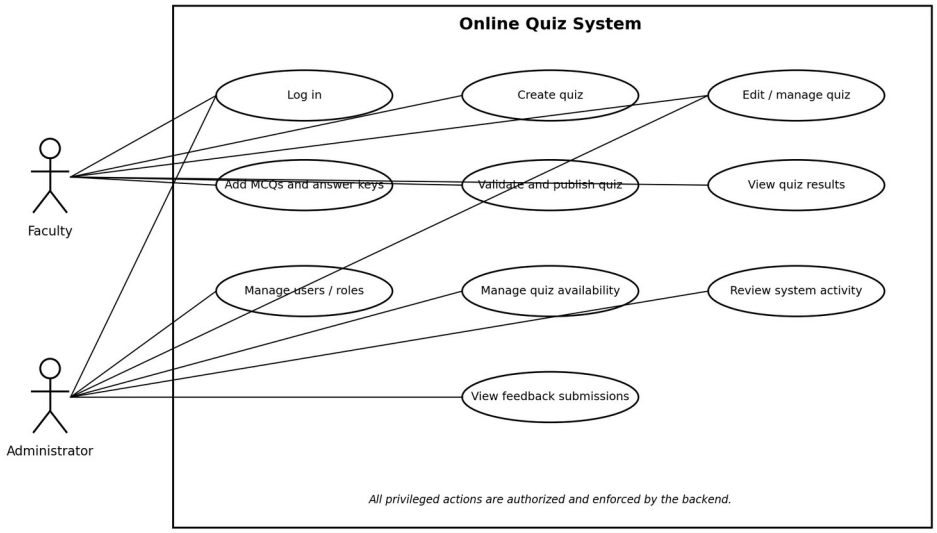


Figure 2. Faculty and administrator interactions: quiz authoring, publication, role management and feedback review.

8. Requirements Traceability Matrix (RTM)

The matrix maps feature groups to verification cases. The detailed requirement identifiers in Section 4 and Section 5 remain the authoritative requirement statements.

Requirement IDs	Feature / quality area	Verification
FR-001–004	Authentication and access	AT-01, AT-06
FR-005–007	Navigation and discovery	AT-10, UI review
FR-008–012	Quiz management	AT-07, API tests
FR-013–018	Quiz attempts and timeout	AT-02, AT-03
FR-019–023	Scoring and leaderboards	AT-04, AT-05
FR-024–026	Feedback and administration	AT-06, API tests
FR-027–031	Version control and CI	AT-08, AT-09
NFR-001–003	Performance and reliability	Load and concurrency tests
NFR-004–007	Usability and maintainability	AT-09, AT-10, README review
SEC-001–006	Security	AT-04, AT-06, security tests and audit

Appendix A. Suggested project structure

client/ (React pages, components and tests); server/ (routes, controllers, models, middleware and tests); .github/workflows/ (CI pipeline); docs/ (API notes, diagrams and evidence); README.md; .env.example.

Appendix B. Team collaboration

All team members should contribute changes across frontend, backend and database. Use small feature branches, descriptive commits, peer-reviewed pull requests and visible CI evidence. Deployment is optional unless explicitly required by the course.